



A
Technical Seminar Report on
Kruskal's and Prim's Algorithms for MST

A9503 – DATA STRUCTURES

B. Tech I year II Semester

Submitted by

25881A0549 – T. SAI PURNANAND

25881A0562 – G. YASHWANTH

25881A0563 – D. YOGESH

Under the guidance of

Dr. V. Lokeshwari Vinya

Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

June 2026

2025-2026

Table of Contents

1. Introduction
2. Overview of the Topic
3. Key Concepts and Technologies
4. Applications and Use Cases
5. Advantages and Challenges
6. Recent Trends or Future Scope
7. Conclusion
8. References

1. Introduction
2. Overview of the Topic
3. Key Concepts and Technologies
4. Applications and Use Cases
5. Advantages and Challenges
6. Recent Trends or Future Scope
7. Conclusion

PPT Slides :

8. References

ABSTRACT

A Minimum Spanning Tree (MST) is a key concept in graph theory that focuses on connecting all vertices of a weighted graph with the minimum total edge cost while avoiding cycles. It is widely used in designing efficient networks such as communication systems, transportation routes, and electrical circuits. The idea is to ensure that all nodes are connected in the most cost-effective way possible.

This topic mainly explores two important greedy algorithms used to find the MST: Kruskal's Algorithm and Prim's Algorithm. Both algorithms aim to minimize the total weight of the spanning tree, but they differ in their approach. Kruskal's algorithm focuses on selecting edges globally, while Prim's algorithm grows the tree from a starting vertex.

The study includes the working principles, algorithmic steps, and examples of both techniques. By implementing these algorithms using appropriate data structures like disjoint sets and priority queues, we can efficiently solve real-world optimization problems.

1. INTRODUCTION

Graph theory plays a significant role in solving complex engineering and computational problems. A graph consists of vertices (nodes) and edges (connections), which may have weights representing cost, distance, or time. In many real-world situations, it is necessary to connect all points with minimum cost, which leads to the concept of a spanning tree.

A spanning tree is a subgraph that connects all vertices without forming any cycles and contains exactly $(V - 1)$ edges. Among all possible spanning trees, the one with the least total edge weight is called the Minimum Spanning Tree (MST). MST ensures optimal connectivity with minimal resource usage.

To find the MST efficiently, greedy algorithms are commonly used. Two of the most popular methods are Kruskal's Algorithm and Prim's Algorithm. These algorithms differ in their approach but ultimately produce the same optimal result.

2. OVERVIEW OF THE TOPIC

The Minimum Spanning Tree is applicable only to connected, weighted, and undirected graphs. The main objective is to include all vertices while minimizing the total edge weight and avoiding cycles. This makes MST an important tool in optimization problems.

Kruskal's Algorithm works by sorting all edges in increasing order of weight and then adding edges one by one to the tree, ensuring no cycle is formed. It treats the graph as a collection of edges and gradually builds the MST.

Prim's Algorithm, on the other hand, starts from an arbitrary vertex and expands the tree by adding the smallest edge that connects a vertex inside the tree to a vertex outside. It focuses more on vertices and grows the tree step by step.

3. KEY CONCEPTS AND TECHNOLOGIES

The key concept behind MST algorithms is the greedy strategy, where the locally optimal choice is made at each step with the hope of finding a global optimum. Important concepts include graph representation, edge weights, cycle detection, and tree construction.

Kruskal’s Algorithm requires sorting of edges and efficient cycle detection, which is typically implemented using the Disjoint Set (Union-Find) data structure. Prim’s Algorithm requires selecting the minimum weight edge efficiently, which is achieved using a priority queue or min-heap.

Both algorithms are implemented using programming languages like C++ with support from data structures such as arrays, heaps, and sets. Their efficiency depends on the data structures used and the size of the graph.

Comparison Table #1

FEATURE	KRUSKAL’S ALGORITHM	PRIM’S ALGORITHM
Approach	Edge-based greedy approach	Vertex-based greedy approach
Starting point	No starting vertex required	Requires a starting vertex
Data structure used	Disjoint Set (Union-Find)	Priority Queue (Min Heap)
Time complexity	$O(E \log E)$	$O(E \log V)$
Mostly used	Sparse graphs (fewer edges)	Dense graphs (more edges)

KRUSKAL'S ALGORITHM

1. First, list all the edges of the graph along with their weights.
2. Sort all the edges in increasing order of their weights.
3. Initialize a forest where each vertex is treated as a separate set.
4. Select the smallest edge from the sorted list.
5. Check whether adding this edge forms a cycle using the disjoint set.
6. If no cycle is formed, include the edge in the MST.
7. If a cycle is formed, discard the edge.
8. Repeat the process until $(V - 1)$ edges are selected.
9. The resulting set of edges forms the Minimum Spanning Tree

KRUSKAL'S ALGORITHM – 9 STEPS EXAMPLE

Example Graph (Undirected Weighted Graph)

Edge	Weight
A - B	4
A - C	1
B - C	2
C - D	4
C - E	3
D - E	2
B - E	5
A - D	1
C - B	2

1 Step 1: List all edges with their weights.

All edges are listed in the table.

2 Step 2: Sort all edges in increasing order of weights.

Sorted Edges	Weight
A - C	1
B - C	2
D - E	2
C - E	3
A - B	4
C - D	4
B - E	5

3 Step 3: Initialize a forest where each vertex is a separate set.

Sets: {A} {B} {D} {E}

4 Step 4: Select the smallest edge from the sorted list.

Smallest edge is A - C (1).

Include A - C.

5 Step 5: Check whether adding this edge forms a cycle.

A and C are in different sets.
No cycle will form.
Include this edge.

6 Step 6: If no cycle is formed, include the edge in the MST.

Edge A - C is included.

7 Step 7: Repeat the process for next edges.

Next edge B - C (2)

Next edge D - E (2)

8 Step 8: Continue until $(V - 1)$ edges are selected.

We have 5 vertices, so we need $(5 - 1) = 4$ edges.

Selected edges: A-C, B-C, D-E, C-E

9 Step 9: The resulting set of edges forms the Minimum Spanning Tree.

Final MST

Edges: A-C (1), B-C (2), D-E (2), C-E (3)

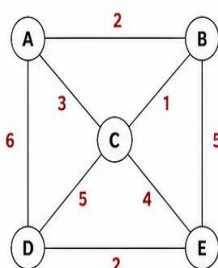
Total Weight = 1 + 2 + 2 + 3 = 8

PRIM'S ALGORITHM

1. Start with any arbitrary vertex of the graph.
2. Mark this vertex as part of the MST.
3. Find all edges connecting this vertex to unvisited vertices.
4. Select the edge with the minimum weight.
5. Add this edge and the connected vertex to the MST.
6. Repeat the process by selecting the smallest edge from all edges connecting visited and unvisited vertices.
7. Continue until all vertices are included in the MST.
8. The collected edges form the Minimum Spanning Tree.

PRIM'S ALGORITHM – 9 STEP EXAMPLE (TREE CONSTRUCTION)

EXAMPLE GRAPH (Undirected Weighted Graph)



EDGE LIST

Edge	Weight
A - B	2
A - C	3
A - D	6
B - C	1
B - E	5
C - D	5
C - E	4
D - E	2

1 Step 1: Start with any arbitrary vertex.

Let's start with vertex A.

MST Set = {A}



Edges from MST to Others	Weight
A - B	2
A - C	3
A - D	6

Choose the minimum weight edge.

2 Step 2: Select the minimum weight edge and add the new vertex.

Minimum edge is A - B (2). Add B to MST.

MST Set = {A, B}



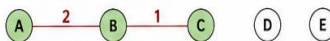
Edges from MST to Others	Weight
B - C	1
A - C	3
B - E	5
A - D	6

Choose the minimum weight edge.

3 Step 3: Select the minimum weight edge and add the new vertex.

Minimum edge is B - C (1). Add C to MST.

MST Set = {A, B, C}



Edges from MST to Others	Weight
C - E	4
C - D	5
B - E	5
A - D	6

Choose the minimum weight edge.

4 Step 4: Select the minimum weight edge and add the new vertex.

Minimum edge is C - E (4). Add E to MST.

MST Set = {A, B, C, E}



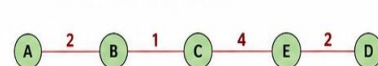
Edges from MST to Others	Weight
E - D	2
C - D	5
A - D	6

Choose the minimum weight edge.

5 Step 5: Select the minimum weight edge and add the new vertex.

Minimum edge is E - D (2). Add D to MST.

MST Set = {A, B, C, E, D}



All vertices are now included in MST.

6 Step 6: Repeat the process of selecting minimum edges.

We repeated until no vertex is left outside MST. At each step, we always picked the smallest edge connecting MST to a new vertex.

7 Step 7: Continue until all vertices are included.

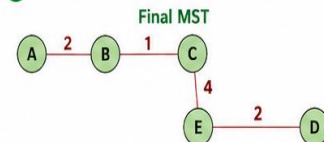
All 5 vertices are now in the MST.

Total edges in MST = $V - 1 = 5 - 1 = 4$

8 Step 8: The edges chosen form the Minimum Spanning Tree.

These edges connect all vertices with minimum total cost without any cycles.

9 Step 9: Final MST and total cost.



Edges in MST: A - B (2), B - C (1), C - E (4), E - D (2)

Total Weight = $2 + 1 + 4 + 2 = 9$

SUMMARY OF PRIM'S ALGORITHM

1. Start with any vertex.
2. Pick the minimum weight edge connecting MST to a new vertex.
3. Add the new vertex to MST.
4. Repeat the process.
5. Continue until all vertices are included.
6. Always choose minimum connecting edge.
7. Total edges in MST = $V - 1$.
8. Edges chosen form the MST.
9. Calculate total weight of MST.

4. APPLICATIONS AND USE CASES

Minimum Spanning Trees are widely used in various real-world applications. In network design, MST helps in connecting computers or communication nodes with the least wiring cost. It is also used in transportation systems to design roads, railways, and pipelines efficiently.

In electrical engineering, MST helps in minimizing wiring in circuit layouts. In computer science, it is used in clustering algorithms for grouping data points in machine learning. Additionally, MST finds applications in image processing and geographical mapping.

These applications highlight the importance of MST in optimizing resources and reducing overall costs in different domains.

5. ADVANTAGES AND CHALLENGES

The major advantage of MST algorithms is that they provide an optimal solution with minimum cost while ensuring connectivity. They are efficient and scalable, making them suitable for large graphs. The greedy approach simplifies the problem-solving process and ensures correctness.

However, there are challenges involved in implementing these algorithms. Kruskal's Algorithm requires efficient cycle detection, which can be complex without proper data structures. Prim's Algorithm requires a good priority queue implementation for better performance.

Additionally, MST algorithms are not suitable for dynamic graphs where edges or vertices change frequently. Their performance also depends on how the graph is represented.

6. RECENT TRENDS OR FUTURE SCOPE

In recent years, MST algorithms have been applied in advanced fields such as machine learning, data mining, and big data analytics. They are used for clustering large datasets and optimizing network structures in smart systems.

With the rise of technologies like IoT and smart cities, MST plays a role in designing efficient communication networks. Future developments include parallel and distributed MST algorithms for handling large-scale graphs in real time.

There is also growing research in dynamic MST algorithms that can handle changes in graphs efficiently, making them more adaptable for modern applications.

7. CONCLUSION

Minimum Spanning Tree is a fundamental concept that provides optimal solutions for connectivity problems. Kruskal's and Prim's algorithms are two effective methods that use greedy strategies to construct MST efficiently.

While Kruskal's algorithm is edge-based and suitable for sparse graphs, Prim's algorithm is vertex-based and works well for dense graphs. Understanding both approaches provides a strong foundation for solving real-world optimization problems.

Overall, MST remains an essential topic in computer science and engineering, with wide applications and future potential.

REFERENCES

1. "Introduction to Algorithms", Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, MIT Press.
2. Data Structures and Algorithm Analysis in C++, Mark Allen Weiss, Pearson Education.
3. Algorithms
Robert Sedgewick and Kevin Wayne, Princeton University.
4. Fundamentals of Data Structures
Ellis Horowitz, Sartaj Sahni, Computer Science Press.
5. Graph Theory Reinhard Diestel, Springer Publications.
6. "GeeksforGeeks" Articles on Minimum Spanning Tree, Kruskal's and Prim's Algorithms.
7. "NPTEL", Data Structures and Algorithms lecture series by IIT professors.
8. "Coursera", Algorithms courses by Stanford University and other institutions.
9. "MIT OpenCourseWare" Lecture notes on Graph Algorithms and MST.